

개인 포트폴리오

바람의 나라

C++ 콘솔 환경에서 구현한 2D MMORPG 모작



박 찬 욱

2016. 12. 25 ~ 2016. 12. 30

1. 개요

넥슨의 2D MMORPG **바람의 나라**를 **C++ 콘솔 프로그램**으로 모작한 작품이다. 게임 엔진이나 그래픽 라이브러리를 일절 쓰지 않고 Win32 API 와 GDI 만으로 타일맵 · 스프라이트 애니메이션 · 전투 · 인벤토리 · 상점 · 대화형 퀘스트까지 하나의 실행 파일에 담았다.

콘솔 창은 본래 문자만 출력하는 환경이다. 이 작품의 출발점은 **그 제약을 우회할 방법을 찾아낸 것으로**, 콘솔 창의 윈도우 핸들을 직접 얻어 GDI 로 비트맵을 그려 넣는 방식을 사용했다.

개발 환경	Visual Studio 2015 (v140) · Win32 Console Application · C++ · 1인 개발
사용 기술	Win32 API, GDI / msimg32 (TransparentBlt), winmm (PlaySound). 외부 엔진 · 라이브러리 없음
개발 기간	6일 (2016. 12. 25 ~ 12. 30)
소스 규모	70개 파일 / 36개 클래스 / 약 3,780줄
리소스	BMP 스프라이트 76장 (원작 화면에서 직접 추출), 애니메이션 60개
구현 범위	타일맵 3종, 몬스터 3종, NPC 2종, 아이템 3종, 7단계 퀘스트 체인

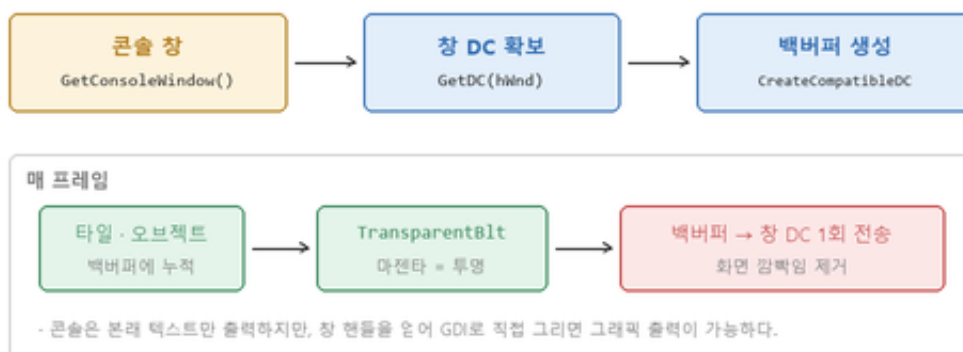
구현 기능

- 콘솔 창에 GDI 로 직접 그리는 렌더링 — 더블 버퍼링과 컬러키 투명 처리
- 타일 격자와 픽셀 좌표를 함께 쓰는 이동, 도착 프레임에 판정을 모으는 구조
- 타일맵에 충돌을 기록하는 예약 방식 충돌 처리
- Port 오브젝트를 통한 맵 간 양방향 이동
- 대화 노드 그래프로 구현한 수집 · 사냥 퀘스트
- 상태 · 방향으로 자동 선택되는 16슬롯 애니메이션, 플레이어 · 몬스터 상태 머신

2. 콘솔 창에 그래픽을 출력하기

콘솔은 문자 단위 출력만 제공한다. 하지만 콘솔 창도 결국 하나의 **윈도우**다. 창이 있다면 그 창을 그릴 수 있는 장치 컨텍스트(DC)도 존재한다는 뜻이다. `GetConsoleWindow()` 로 창 핸들을 얻고 `GetDC()` 로 DC 를 확보하면, 그 시점부터는 일반 Win32 GUI 프로그램과 다를 게 없다.

그림 1. 콘솔 창을 그래픽 서피스로 사용하는 렌더링 파이프라인



타일 · 오브젝트 · UI 를 창 DC 에 하나씩 직접 그리면 그리는 과정이 그대로 보여 심하게 깜빡인다. 그래서 창과 호환되는 메모리 DC 와 비트맵을 **백버퍼**로 쓰고, 한 프레임의 모든 그리기를 백버퍼에 누적한 뒤 마지막에 **한 번만** 창 DC 로 옮긴다. 또 BMP 는 알파 채널이 없으므로 스프라이트 배경을 **마젠타(255, 0, 255)**로 칠해 두고, `TransparentBlt` 에 그 색을 컬러키로 넘겨 해당 픽셀을 건너뛰게 했다.

```
system ("mode con:cols=53 lines=18"); // 창 크기를 문자 격자로 지정
SetConsoleCursorInfo (...); // 깜빡이는 커서 제거

gHwnd = GetConsoleWindow (); // 콘솔 창의 윈도우 핸들
gHDC = GetDC (gHwnd); // 그 창에 그릴 수 있는 DC

// 스프라이트 출력 — 마젠타를 투명색으로 지정
TransparentBlt (_hBackDC, x + offsetX, y + offsetY, w, h, _hMemDC, 0, 0, w, h,
    RGB (255, 0, 255));
```

그리기 순서는 뒤에서 앞으로 쌓는 방식을 그대로 따랐다. 타일 → 아이템 → 몬스터 → 플레이어 → NPC → 장식물 → UI → 대화창 순으로 그려서, 깊이 정렬 없이도 캐릭터가 타일 위에 서고 대화창이 항상 맨 위에 오도록 했다.

3. 리소스 — 원작 화면에서 스프라이트 추출

모작이므로 사용할 그래픽이 따로 없었다. 원작을 실행해 화면을 캡처하고 캐릭터 · 몬스터 · 타일 · UI 프레임들 **한 장씩 직접 잘라내** BMP 로 정리했다. 아틀라스 도구 없이 만든 리소스라 스프라이트마다 기준점이 제각각이었는데, 이 불규칙함은 **이미지를 등록하는 시점에 오프셋을 함께 저장하는** 방식으로 흡수했다. 그리기 함수가 좌표에 오프셋을 자동으로 더하므로 호출부는 타일 좌표만 넘기면 된다. 스프라이트의 발끝이 24px 격자에 맞도록 Y 오프셋을 음수로 밀고, 좌우로 크게 뻗는 공격 모션은 X 오프셋을 음수로 줘서 무기가 옆 타일로 자연스럽게 넘어가게 했다.

그림 7. 원본 게임 스크린샷에서 스프라이트를 직접 추출



- 게임 엔진의 스프라이트 에디터나 아틀라스 도구 없이, 캡처 화면에서 프레임들 한 장씩 잘라 BMP 로 정리했다.
- 알파 채널이 없는 BMP 이므로 배경을 마젠타로 칠하고, `TransparentBlt` 의 컬러키로 지정해 투명 처리했다.
- 스프라이트마다 기준점이 다르므로 등록 시점에 오프셋을 함께 넣어, 24px 격자 위에 발끝이 맞도록 보정했다.
- 총 76 장의 BMP, 애니메이션 60 개, 이미지 키 68 개를 등록해 사용했다.

4. 타일 단위 이동과 픽셀 단위 보간

원작은 타일 기반이지만 캐릭터가 타일 사이를 **미끄러지듯** 이동한다. 타일 단위로 순간이동하면 로직은 간단하지만 움직임이 끊기고, 픽셀 단위로만 다루면 충돌과 상호작용 판정이 복잡해진다. 두 좌표계를 동시에 유지하는 방식으로 풀었다 — 모든 오브젝트가 `_worldX/_worldY` (타일 격자) 와 `_localX/_localY` (픽셀) 를 함께 갖는다.

그림 2. 이중 좌표계와 타일 단위 이동



격자 단위로 판정하고 픽셀 단위로 보여주기 때문에, 로직은 타일맵처럼 단순하면서 화면은 부드럽게 움직인다.
 타일 정중앙에 도달한 프레임에서만 맵 이동 · 다음 입력 · 상태 전환을 판정하므로, 이동 중간 상태가 끼여들 여지가 없다.
 방향키를 계속 누르고 있으면 도착 프레임에서 곧바로 다음 타일을 예약해 연속 이동이 끊기지 않는다.

이동은 프레임마다 픽셀 좌표를 1 씩 옮기는 것으로 끝이다. 중요한 건 **언제 도착으로 볼지**인데, 픽셀 좌표가 타일 크기로 정확히 나누어떨어지는 순간이 곧 타일 정중앙이다.

```
_localX += _dirX;
_localY += _dirY;

if (_localX % 24 == 0 && _localY % 24 == 0) // 타일 정중앙에 도달
{
    // 이 프레임에서만 맵 이동 · 다음 입력 · 상태 전환을 판정한다
    Port* port = currentMap->CheckPort (_worldX, _worldY);
    ...
    if (방향키가 눌러 있으면) KeyInputWithMoveState (...); // 다음 타일 예약
    else state = IDLE;
}
```

이 구조에서 얻은 것

- 판정 지점이 한 곳으로 모인다. 맵 이동 · 입력 재판정 · 상태 전환이 모두 '타일 정중앙에 도달한 프레임'에서만 일어나므로, 이동 중간 상태가 끼여들 여지가 없다.
- 연속 이동이 끊기지 않는다. 도착 프레임에서 방향키가 여전히 눌러 있으면 그 자리에서 다음 타일을 예약하고 이동을 이어받는다. 타일마다 멈칫하는 느낌이 나지 않는다.

5. 타일맵에 충돌을 기록하는 예약 방식 충돌 처리

오브젝트끼리 짝지어 겹침을 검사하면 개체 수가 늘 때 비용이 제곱으로 커진다. 타일 기반이라면 한 타일에 하나만 서 있을 수 있으므로, **충돌 정보를 오브젝트가 아니라 타일이 들고 있게** 하면 갈 곳의 타일 하나만 보면 된다.

그림 3. 타일에 충돌을 기록하는 예약 방식 충돌 처리



여기서 문제가 하나 생긴다. 픽셀 단위로 이동하는 동안 캐릭터는 **두 타일 사이에 걸쳐** 있다. 아직 도착하지 않았으니 그 사이에 다른 몬스터가 같은 타일로 들어와 겹칠 수 있다. 그래서 이동을 **시작하는 순간** 목표 타일을 미리 충돌 상태로 예약한다. 반대로 예약만 하고 해제를 빠뜨리면 지나온 자리마다 **유령 충돌**이 남아 맵이 점점 막히므로, 모든 오브젝트가 매 프레임 자기가 밟은 타일을 확인해 이전 타일을 되돌리는 공통 루틴을 `Object` 에 뒀다.

```
// 이동 시작 — 목표 타일을 미리 막아 둔다
if (moveTile && moveTile->GetCollider () != Object::COLLISION)
{
    state = MOVE;
    moveTile->SetCollider (Object::COLLISION); // 예약
}
else state = IDLE; // 막혀 있으면 제자리

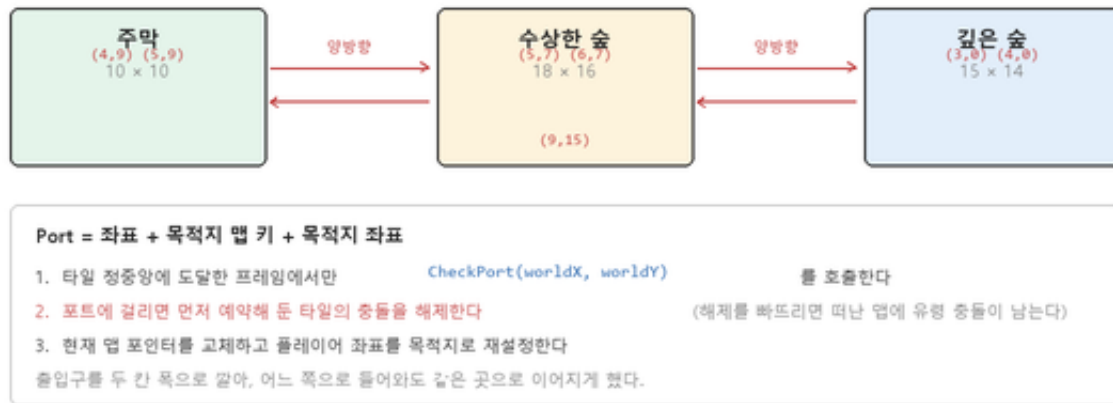
// 매 프레임 — 밟고 있는 타일이 바뀌었으면 이전 타일을 원복
if (_stepTile != NULL && _stepTile != 현재타일)
    _stepTile->SetCollider (Tile::NORMAL); // 해제
```

덕분에 판정 비용이 $O(1)$ 이고, 벽 · 캐릭터 · NPC 를 같은 방식으로 다룰 수 있다. 몬스터의 배회 로직도 이 정보를 그대로 써서 네 방향 중 지나갈 수 있는 방향만 후보로 모으므로, 벽으로 돌진하는 움직임이 애초에 나오지 않는다.

6. Port 를 통한 맵 전환

맵은 각자 타일 배열과 몬스터 · NPC · 장식물 목록을 갖는 독립 객체이고, `MapManager` 가 이름을 키로 들고 있다가 현재 맵 포인터만 갈아끼운다. 맵 사이를 잇는 통로는 좌표 하나에 **목적지 맵 이름**과 **목적지 좌표**를 붙인 `Port` 오브젝트로 표현했다.

그림 4. Port 오브젝트를 통한 맵 전환

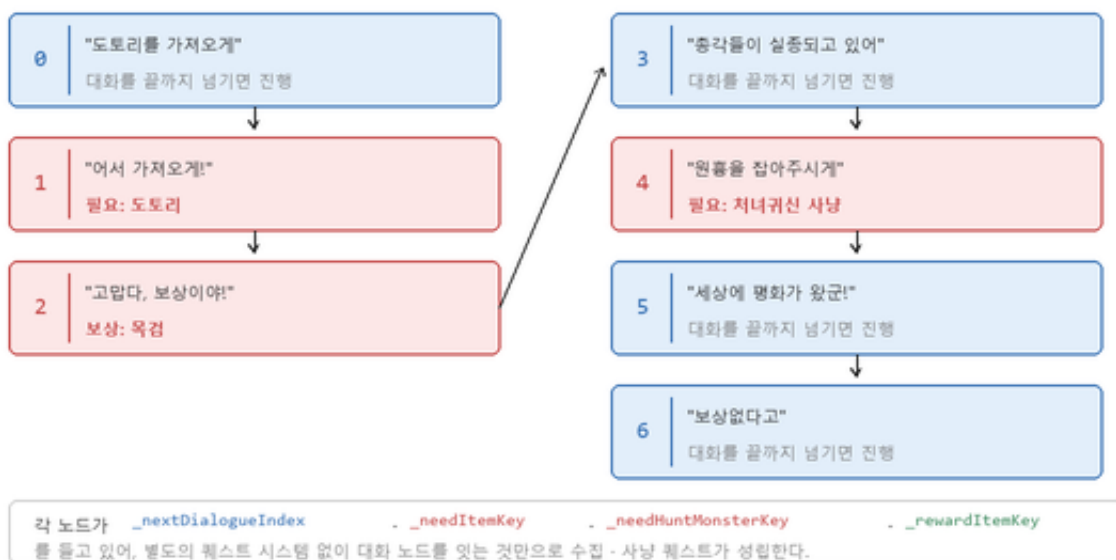


포트 판정은 타일 정중앙 도달 프레임에서만 일어난다 — 걸쳐 있는 상태로 판정하면 한 번의 이동으로 두 번 전환되는 문제가 생긴다. 그리고 전환 직전에 **예약해 둔 타일의 충돌을 반드시 해제**해야 한다. 빠뜨리면 떠난 맵에 지나갈 수 없는 칸이 남는데, 실제로 겪은 버그였다. 출입구는 **두 칸 폭**으로 깔아 두 좌표에 각각 포트를 두고 같은 목적지로 보냈다 — 한 칸짜리 문은 정확히 그 칸을 밟아야 해서 조작이 답답해지기 때문이다.

7. 대화 노드 그래프로 구현한 퀘스트

퀘스트 시스템을 따로 만들지 않고 **대화 그 자체를 상태로** 다뤘다. NPC 는 대화 목록과 '지금 몇 번째 대화인지' 를 가리키는 인덱스를 들고 있고, 각 노드는 다음에 갈 노드 번호와 진행 조건(`_needItemKey` · `_needHuntMonsterKey`), 보상(`_rewardItemKey`)을 갖는다. 노드를 잇는 것만으로 수집 퀘스트와 사냥 퀘스트가 성립한다.

그림 5. 대화 노드 그래프로 구현한 퀘스트 진행



조건이 걸린 노드는 **조건이 충족되는 순간 자기를 건너뛰고** 다음 노드를 대신 띄운다. 덕분에 "아직 못 가져왔군" 과 "고맙다, 보상이야" 를 분기문 없이 두 개의 노드로 표현할 수 있다. 사냥 조건은 같은 방식으로 플레이어가 들고 있는 사냥 기록 목록을 조회한다 — 몬스터가 죽을 때 자기 이름을 그 목록에 넣어두기 때문에, 퀘스트 쪽에서는 별도의 이벤트 연결 없이 물어보기만 하면 된다.

```

if (_needItemKey != "") // 필요아이템이 있는 경우
{
    for (인벤토리의 각 아이템)
        if (_needItemKey == 아이템 이름)
        {
            인벤토리에서 제거; // 퀘스트 아이템 소모
            npc->SetCurrentDialogue (_nextDialogueIndex); // 다음 노드로 교체
            _enabled = false;
            return;
        }
}

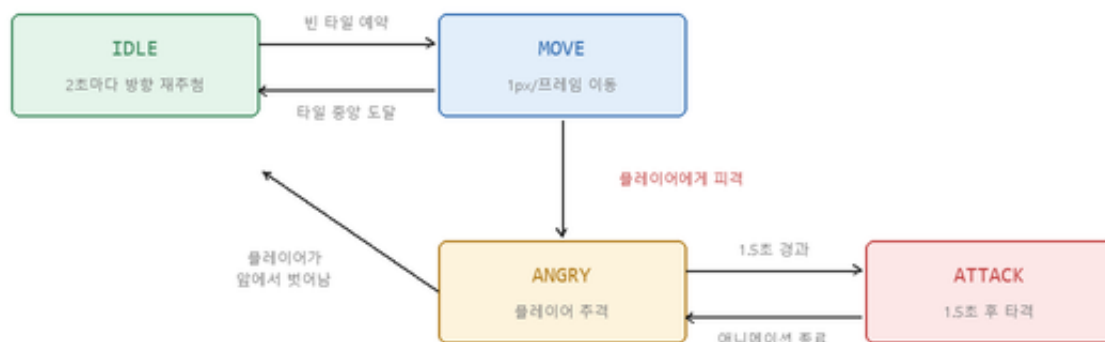
```

NPC 는 맵마다 별도의 인스턴스로 배치되지만 퀘스트 진행도는 한 곳에만 있어야 한다. 그래서 대화 상태는 맵의 인스턴스가 아니라 **NpcManager** 가 들고 있는 **원본 데이터**에 저장하고, 맵의 인스턴스는 이름으로 원본을 찾아가 대화를 갱신 · 출력한다. 같은 NPC 를 여러 맵에 배치해도 진행도가 갈라지지 않는다.

8. 상태 머신과 애니메이션

플레이어와 몬스터는 모두 **switch** 기반 유한 상태 머신으로 동작한다. 플레이어는 **IDLE · MOVE · ATTACK · FARMING · OTHER_ACTION** 다섯 상태, 몬스터는 **IDLE · MOVE · ANGRY · ATTACK** 네 상태를 갖는다.

그림 6. 몬스터 상태 머신



- IDLE 은 네 방향 중 지나갈 수 있는 타일만 골라 무작위로 하나 뽑는다 — 벽에 저박히는 움직임이 나오지 않는다.
- ANGRY 는 플레이어 방향을 구해 부호만 남기고, 대각선이 나오면 무효로 돌린다 (타일맵은 4방향만 허용).

몬스터 추격은 플레이어와의 좌표 차에서 부호만 남겨 방향을 구하는데, 이러면 대각선이 나올 수 있다. 타일맵에서는 4방향만 허용되므로 **대각선이 나온 경우는 무효로 돌려** 배회 상태로 되돌린다. 공격은 마주한 채 1.5초가 지나면 들어가고, 타격 판정은 공격 애니메이션이 **끝나는 프레임**에서 이뤄지며 그 순간에도 플레이어가 앞에 있는지 다시 확인한다 — 선딜 중에 피해 범위를 벗어나면 맞지 않는다.

캐릭터는 (상태 × 방향) 조합에 대응하는 16개의 애니메이션 슬롯을 갖는다. 애니메이션을 코드 곳곳에서 직접 바꾸면 상태와 화면이 어긋나기 쉬우므로, **매 프레임 끝에 현재 상태와 방향으로부터 슬롯을 결정**하도록 한 곳에 모았다. 이동 · 공격 · 채집 로직은 애니메이션을 전혀 신경 쓰지 않고, 애니메이션 종료 플래그가 그대로 상태 전이 조건이 되므로 연출과 판정 타이밍이 자동으로 일치한다.

9. 회고

가장 값진 경험은 **환경의 제약을 우회하는 방법을 스스로 찾아낸** 것이었다. "콘솔에서는 그림을 못 그린다" 를 전제로 받아들이지 않고, 콘솔 창도 윈도우라는 사실에서 출발해 창 핸들 → DC → GDI 라는 경로를 찾아낸 과정이 이 작품의 출발점이자 핵심이다.

설계 측면에서는 **판정 지점을 한 곳으로 모으는 것**의 효과를 체감했다. '타일 정중앙에 도달한 프레임' 하나에 맵 이동 · 입력 재판정 · 상태 전환을 모두 묶어두니, 이동 중 애매한 상태에서 생기는 버그가 애초에 발생하지 않았다. 충돌을 타일에 기록하는 방식도 마찬가지로, 벽과 캐릭터와 NPC 를 같은 규칙으로 다룰 수 있게 해줬다.

반면 지금 다시 만든다면 고칠 부분도 분명하다. 맵 · 몬스터 · 퀘스트 데이터가 모두 **코드에 하드코딩**되어 있어 내용을 바꿀 때마다 재컴파일이 필요했다. 외부 데이터 파일로 분리했다면 훨씬 빠르게 반복할 수 있었을 것이다. 프레임 제한이 없어 이동 속도가 실행 환경의 성능에 좌우되는 것도 당시에는 미처 신경 쓰지 못한 부분이다.

클래스 구성

Object 좌표 · 방향 · 충돌 · 밟은 타일 갱신 (모든 오브젝트의 기반) · **Character** 상태 머신 · 스택 · 애니메이션 선택

Player / **Monster** / **Npc** 각자의 상태 머신과 행동 · **Tile** / **Port** / **Item** / **Ornament** 맵 구성 요소

Map 타일 배열과 배치 오브젝트 · **RenderManager** 이미지 등록 · 백버퍼 · 출력

매니저 6종 — 원본 데이터를 문자열 키로 보관하고 복사 생성으로 인스턴스를 만드는 프로토타입 방식

Time 다중 독립 타이머 믹스인 (입력 연타 억제 · 몬스터 판단 주기 · 대화 넘김 디바운스) · **Singleton** 템플릿

Windows 11 에서 실행할 때

기본 콘솔 호스트(Windows Terminal)는 콘솔 창에 GDI 로 직접 그리는 방식을 지원하지 않아 화면이 검게만 나온다. 레거시 **conhost.exe** 를 통해 실행하면 정상적으로 출력된다.

```
conhost.exe 161225_Baram.exe
```